

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 6

Inside the xv6 File System

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: Read What xv6 Shows You 2
2.1.1 Observe Answers (Part 1) 3
2.2 Part 2: Write `mystat` 4
2.2.1 Observe Answers (Part 2) 4
2.3 Part 3: Measure File System Limits 5
2.3.1 Observe Answers (Part 3) 5
2.4 Part 4: Write `find` 7

3 CONCLUSIONS **8**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The goal of this laboratory work is to explore the internal architecture of the xv6 file system, implement user-space utilities in C (`mystat` and recursive `find`), understand hard link mechanisms at the kernel level, and measure structural limits defined in `kernel/fs.h`.

1.2 Theoretical Background

The xv6 file system implements a Unix-like structure structured in layers:

- **Disk Layer & Superblock:** Defines total disk blocks, log block allocations, and inode block counts.
- **Inode Layer:** Each file is represented on disk by a `struct dinode` holding metadata (type, size, link count) and block pointers (12 direct blocks + 1 indirect block).
- **Directory Layer:** Directories are special files containing array entries of `struct dirent` (fixed 16-byte size: 2-byte inode number + 14-byte name array `DIRSIZ`).
- **System Calls:** `stat()` retrieves metadata into `struct stat`, while `sys_unlink()` decrements hard link counters, releasing data blocks when link count drops to zero.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: Read What xv6 Shows You

Running `ls` in xv6 outputs entries formatted as `name, type, inode, size`.

```
hart 2 starting
hart 1 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
mystat         2 23 36344
find           2 24 42720
console        3 25 0
$ mkdir a
$ ls a
.              1 26 32
..             1 1 1024
$
```

Figure 2.1: Inspection of default xv6 directory entries and types

2.1.1 Observe Answers (Part 1)

1. **Type Codes (`kernel/stat.h`):**
   - `1` = `T_DIR` (Directory)
   - `2` = `T_FILE` (Regular File)
   - `3` = `T_DEVICE` (Device, e.g. console)
2. **Root Directory Inode:** The root directory `/` has inode number **1**. The entries `.` and `..` have identical inode numbers (`1`) because the root directory has no parent above it, so `..` points back to itself.
3. **Directory Entry Size:** Each `struct dirent` occupies $2 + 14 = 16$ bytes. An empty directory `a` contains 2 entries (`.` and `..`), resulting in a size of $2 \times 16 = 32$ bytes.

2.2 Part 2: Write `mystat`

A custom utility `user/mystat.c` was created to format file metadata retrieved via `stat()`.

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 25, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Figure 2.2: Execution of mystat with hard link creation and deletion

2.2.1 Observe Answers (Part 2)

1. **Hard Link Output Changes:**
   - After `ln README r2`: Link count increased from 1 to 2 for inode 2. The size (2441 bytes) and inode number remained identical for both names.
   - After `rm README`: Link count decreased from 2 to 1 for `r2`. The file content remained fully accessible.
2. **Unlink Implementation in Kernel:** Line `ip->nlink-;` in `kernel/sysfile.c` decrements link count. Data blocks are freed inside `iput()` when `nlink == 0` and no active process holds a reference.
3. `struct dinode` **Fields (**`kernel/fs.h`**):** Contains `type`, `major`, `minor`, `nlink`, `size`, and `addrs[NDIRECT+1]`.

2.3 Part 3: Measure File System Limits

System constants in `kernel/fs.h` establish physical disk boundaries.

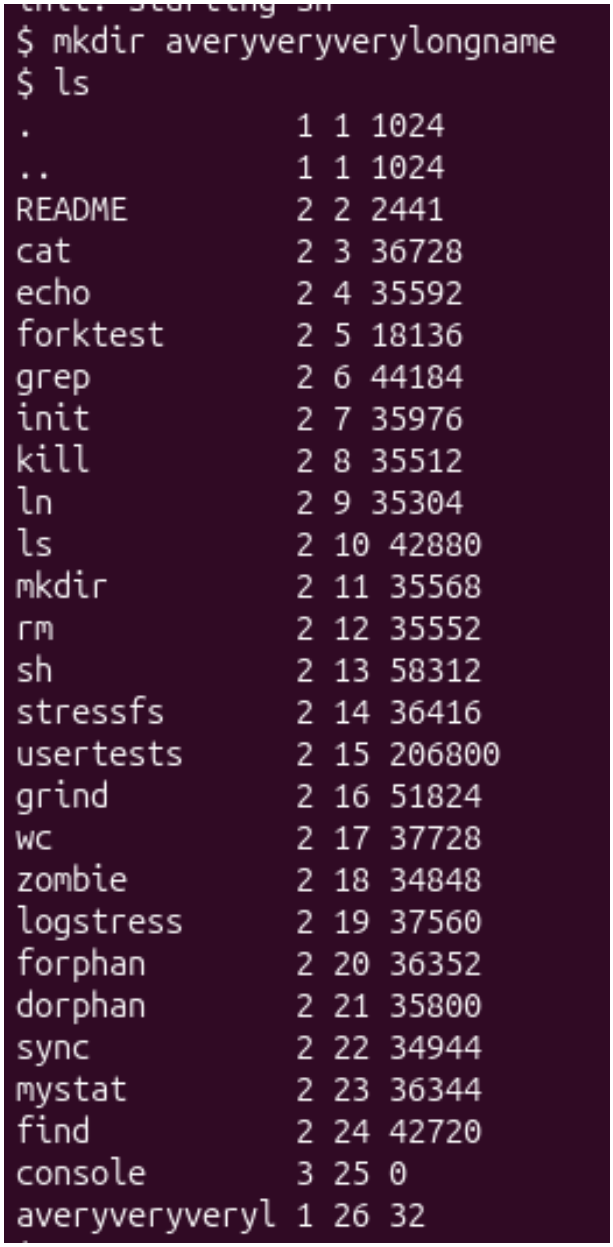


Figure 2.3: Inspection of kernel limits, disk layout, and filename truncation

2.3.1 Observe Answers (Part 3)

1. **Filename Truncation:** `mkdir averyveryverylongname` displays as `averyveryverylo` because `#define DIRSIZ 14` limits filenames to 14 bytes.

2. **Maximum File Size Calculation:**

$$\text{Direct Blocks} = 12 \times 1024 = 12,288 \text{ bytes (12 KB)}$$

$$\text{Indirect Blocks} = \frac{1024}{4} = 256 \text{ blocks} \rightarrow 256 \times 1024 = 262,144 \text{ bytes (256 KB)}$$

$$\text{Max Size} = (12 + 256) \times 1024 = 274,432 \text{ bytes} (\approx 268 \text{ KB})$$

3. **Disk Layout Metadata:** Out of 2000 total blocks (2 MB), metadata uses 46 blocks (1 boot, 1 superblock, 30 log, 13 inodes, 1 bitmap), leaving 1954 data blocks.

2.4 Part 4: Write `find`

The utility `user/find.c` recursively traverses directory trees, skipping `.` and `..` entries.

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
$
```

Figure 2.4: Recursive search execution with find utility

3 CONCLUSIONS

In this laboratory work, I explored the internal file system mechanics of the xv6 operating system. By implementing `mystat.c`, I gained a practical understanding of how system calls expose raw inode metadata and how hard links manage file lifetime independently of filenames.

By analyzing `kernel/fs.h`, I derived the exact mathematical limits of the xv6 file system architecture, including filename constraints (`DIRSIZ`) and maximum file sizes dictated by single indirect block addressing. Finally, developing the recursive `find.c` utility reinforced directory reading patterns, string formatting, and resource cleanup (descriptor closing) within a minimal OS environment.